Open Access | https://doi.org/10.25080/fdgu8736



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Avoiding Zero-Trade Policies in RL with a Decoupled MLOps Architecture

Sho Tanaka[1]

[1]Snowflake, Tokyo

Abstract

Reinforcement learning (RL) agents trained on financial time series frequently end up with degenerate policies that never complete a trade. We report on a series of experiments with a Deep Q-Network (DQN) trading USD/JPY 5-minute bars, built with Stable-Baselines3 and Gymnasium, in which the direct end-to-end baseline produced 0 completed trades and 0.00% realized return on the held-out period. We then describe the architectural response presented at the SciPy 2026 virtual poster session: the RL model acts as a *signal generator* that emits a directional score in $[-1, +1]$, and a deterministic, unit-testable Python *execution engine* owns position management—ATR stop-loss, risk-reward take-profit, trailing stop, time-based exit and a risk manager. On the same data and split, the decoupled system completed 2,625 fully risk-managed trades, with a 48.4% win rate and a realized return of -0.64%. `EvalCallback` retained the best evaluated checkpoint rather than relying on the final training state. The result demonstrates the operational value of separating signal generation from deterministic execution; it is a behavioral backtest on one currency pair, not a claim of profitability.

Keywords Reinforcement Learning, MLOps, FinTech

1. Introduction

Reinforcement learning is an attractive framing for automated trading: an agent observes market features and chooses among *buy*, *sell* and *hold* actions, and a reward signal derived from profit-and-loss (PnL) is supposed to shape a policy that adapts to market regimes without hand-written entry and exit rules [1], [2], [3]. The open-source Python stack makes this easy to try—Gymnasium [4] for the environment interface, Stable-Baselines3 [5] for the algorithms, NumPy [6] and pandas [7] for feature engineering, and TA-Lib [8] for a subset of standard technical indicators.

In practice, it is easy to build an agent that *appears* to train for hours and then does nothing at all. Over roughly 75 experiments on USD/JPY 5-minute data, our DQN and PPO runs repeatedly ended with policies that completed zero trades on the held-out period. The poster framed this as a reward-design trap: every completed trade pays a transaction cost, holding costs nothing, and "never trade" can therefore become an attractive policy.

This paper expands the content presented as a virtual poster at SciPy 2026: the zero-trade baseline and a decoupled architecture in which the RL model only emits a directional score while a deterministic Python execution engine owns all position management. On the same data and split, the decoupled system trades (2,625 completed trades) where the direct agent does not (0 trades).

Published Jul 13, 2026

Correspondence to
Sho Tanaka
sho.tanaka@snowflake.com

Open Access

Copyright © 2026 Tanaka. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

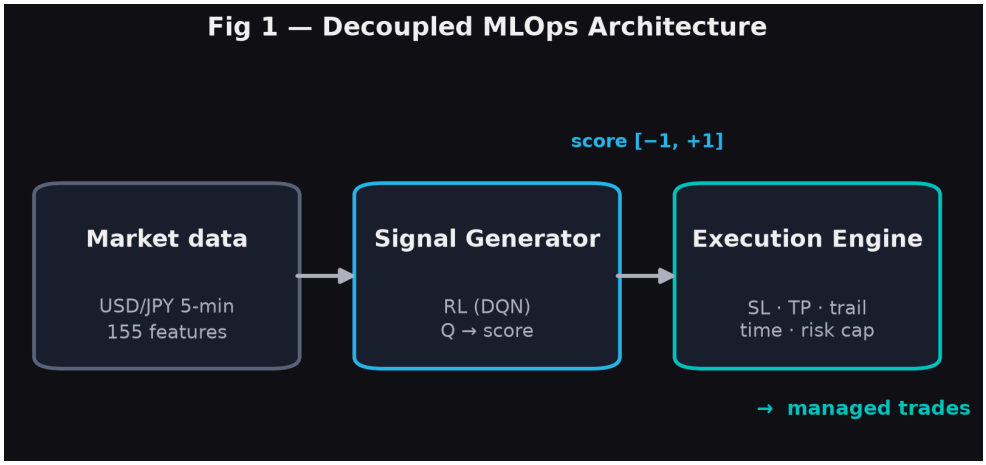


Figure 1. *The decoupled architecture. Market features are passed to the DQN signal generator, and only its directional score crosses the boundary to the deterministic execution engine, which owns exits and risk controls.*

Our contributions are:

- A reproducible zero-trade baseline on public-format FX data with a Stable-Baselines3 DQN, and the reward variants we tried before abandoning reward shaping (Section Experimental Setup and the Zero-Trade Baseline).
- A decoupled signal/execution architecture with concrete Python patterns, deterministic risk controls and checkpoint selection with `EvalCallback` (Sections Architecture: Decoupled Signal Generation and Execution and Results Presented at the Poster).
- A same-data comparison showing 0 completed trades for the direct baseline and 2,625 fully managed trades for the decoupled system.

We make no claim of profitability. All PnL figures are backtests on a single currency pair and are reported to characterise *behavior*, not returns.

2. Experimental Setup and the Zero-Trade Baseline

2.1. Data and features

All experiments use USD/JPY 5-minute OHLC bars. The poster experiments use the period 2025-01-10 through the last available bar on 2025-12-19 (70,555 bars after feature warm-up). The poster reports the calendar endpoint as 2025-12-20. We use a chronological 70/30 train/test split (49,388 / 21,167 bars). The 21,167 test bars yield 21,116 evaluable steps after constructing 50-bar observations and next-bar transitions. A `FeatureEngineer` computes 155 technical features per bar (moving averages, RSI, MACD, ATR, Bollinger bands, returns at several horizons, and so on). TA-Lib is a Python wrapper around the widely used open-source TA-Lib C library, which implements over 150 standard technical analysis indicators; it provides a subset of the indicators used here, and the remaining custom and return-based features are implemented with NumPy and pandas. The observation comprises a 50-bar window, flattened to a $50 \times 155 = 7,750$-dimensional vector.

2.2. Environment and agent

The environment is a Gymnasium `Env` with a discrete action space $\{\mathrm{HOLD}, \mathrm{BUY}, \mathrm{SELL}\}$. BUY opens or holds a long position, SELL opens or holds a short position, and switching direction closes the existing position first. A transaction cost of 1 pip (equal to 0.0001 in this environment's price units) is charged each time a position is opened or closed, so a

completed round-trip pays 2 pips. Episodes are 1,000 bars long with random start points. The reward at each step is

$$r_t = 100 \cdot \Big(w_u \, \Delta\mathrm{PnL}_t^{\mathrm{unrealized}} + \Delta\mathrm{PnL}_t^{\mathrm{realized}} + b_{\mathrm{complete}} \, 1[\text{trade closed}] + b_{\mathrm{profit}} \, 1[\text{trade closed with profit}] \Big) \tag{1}$$

where w_u, b_{complete} and b_{profit} are the reward design knobs we varied. Writing p_t for the close of bar t and $\pi_t \in \{-1, 0, +1\}$ for the position after the action at step t (short, flat, long), the unrealized term is the one-bar fractional price change signed by the open position, $\Delta\mathrm{PnL}_t^{\mathrm{unrealized}} = \pi_t \, (p_{t+1} - p_t)/p_t$ (zero when flat), and the realized term is zero except on a step that closes a position opened at price p_{entry} with direction π, when $\Delta\mathrm{PnL}_t^{\mathrm{realized}} = \pi \, (p_t - p_{\mathrm{entry}})/p_{\mathrm{entry}}$. The transaction cost is subtracted from the reward each time a position is opened or closed (omitted from (1) for readability). The agent is a Stable-Baselines3 `DQN` [9] with an MLP policy (`net_arch=[512, 512, 512, 256]`), learning rate 10^{-4}, replay buffer 200k, batch size 512, $\gamma = 0.99$, ε-greedy exploration annealed over the first 15% of training to a floor of 0.05, 100k training steps, CPU only. Evaluation is deterministic (greedy argmax) over the full test period.

2.3. Reward variants tried before the poster

Table 1 summarises the reward designs explored during development, labeled as on the poster, and Figure 2 visualises their trade counts. The counts are from the original single-seed development runs: each configuration was trained exactly once, with one random-number seed governing network initialization, exploration and episode start points. Because run-to-run variability under different seeds was not measured, the counts are indicative only.

Table 1. *Reward designs explored during development and their test-set trade counts (single-seed development runs).*

Label	Reward configuration	Completed trades	Observed behaviour
Buy & Hold	unrealized PnL rewarded ($w_u = 1$)	24	Enters long early, rarely exits
Penalty Hell	hold penalty added	58	Trades, but erratically
Zero-Trade Collapse	$w_u = 0$, no bonuses, 1 pip cost	**0**	Single constant action
Trade Bonus	$b_{\mathrm{complete}} > 0$, $b_{\mathrm{profit}} > 0$	84	Trades, unstable across runs

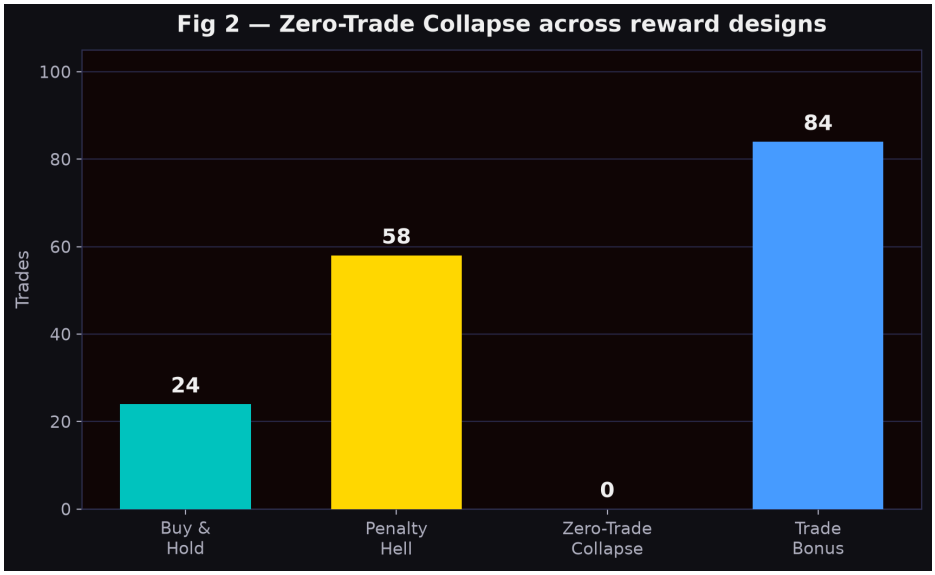


Figure 2. *Completed test-set trades for the four reward designs in Table 1. These counts come from single-seed development runs and show that some reward variants induced trading, but not that they were stable or profitable.*

The "Zero-Trade Collapse" configuration ($w_u = 0$, $b_{\text{complete}} = b_{\text{profit}} = 0$) is the baseline used throughout the rest of the paper. The configuration emits BUY on all 21,116 test steps and completes 0 trades, for a realized return of 0.00% when reproduced on the 2025 period with the aforementioned settings. A policy that emits BUY forever opens one position on the first bar and never closes it, so it registers as zero *completed* trades and zero *realized* PnL.

2.4. Why reward shaping was abandoned

Each reward variant that produces trades does so at the price of a new hyperparameter (b_{complete}, b_{profit}, the hold penalty) whose value is tuned to the training period and does not transfer. The reward is asked to simultaneously encourage *good* trades, discourage *bad* ones, and define what "good" means, and every adjustment moves the equilibrium—by which we mean the stable policy that training settles into and no longer improves away from under a given reward design—rather than removing the degenerate one. Each row of Table 1 (buy-and-hold, erratic trading, a single constant action) is such an equilibrium. After roughly 75 such runs, we stopped editing the reward and changed the system boundary instead.

3. Architecture: Decoupled Signal Generation and Execution

3.1. Design

The monolithic agent combines directional prediction and position management in a single discrete action. "Decoupling" here means splitting the system at exactly that point: the learned model and the hand-written execution logic become two independent components whose only interface is a single scalar score, so that either side can be tested, inspected or replaced without touching the other. The decoupled system assigns these responsibilities to separate components:

- **Directional scoring** is handled by the RL model, which emits a continuous score $s_t \in [-1, +1]$.
- **Position management** is handled by a deterministic Python execution engine with no learned parameters.

The high-level data flow is shown in Figure 1.

The model is trained in a *market-only* variant of the environment whose observation excludes the agent's own position state and whose reward (`direction_reward_weight = 1.0`, with small completion and profit bonuses of 5×10^{-4} and 3×10^{-4}) rewards calling the next bar's direction correctly. At inference the model is never asked to act; its Q-values are read off and converted to a score by a `DQNScoreSignalGenerator`:

```
class DQNScoreSignalGenerator(SignalGenerator):
    HOLD, BUY, SELL = 0, 1, 2

    def generate(self, observation, **_):
        obs = torch.as_tensor(observation[None], device=self.model.device)
        with torch.no_grad():
            q = self.model.q_net(obs)[0].cpu().numpy()
        score = float(np.tanh((q[self.BUY] - q[self.SELL]) / self.temperature))
        return TradingSignal(score=score, confidence=abs(score))
```

The execution engine consumes the score through a configuration object that states every rule explicitly. The configuration used for all decoupled runs in this paper is:

```
ExecutionConfig(
    signal=SignalThresholdConfig(entry_threshold=0.2, exit_threshold=0.1,
                                 reversal_threshold=0.5),
    stop_loss=StopLossConfig(type=StopLossType.ATR_BASED, atr_multiplier=1.0),
    take_profit=TakeProfitConfig(type=TakeProfitType.RISK_REWARD,
                                 risk_reward_ratio=2.5),
    trailing_stop=TrailingStopConfig(type=TrailingStopType.ATR_BASED,
                                     activation_profit_pips=5.0,
                                     atr_multiplier=0.5),
    time_exit=TimeExitConfig(enabled=True, max_bars_in_trade=48),
)
```

A score with $|\ s_t\ | \geq 0.2$ opens a position in the direction of the sign; the engine then attaches a stop-loss one ATR away, a take-profit at 2.5 times the stop distance, a trailing stop that activates after 5 pips of profit, and a hard time exit after 48 bars (four hours). A `RiskManager` enforces daily loss and drawdown caps, and open positions are closed before the weekend.

3.2. Core classes

The execution layer is organised around a small number of plain dataclasses and one stateful manager. The pattern is the one named in the original poster abstract (`Position` / `Order` / `RiskManager` / `ExitPolicy`); in the code that produced the results, the exit policy is split into per-rule `*Config` objects applied by a `PositionManager`, and the `RiskManager` enforces `max_position_size`, `max_daily_loss` (2%) and `max_drawdown` (10%) of balance.

```
class PositionSide(Enum):
    LONG = 1
    SHORT = -1

class ExitReason(Enum):
    SIGNAL = "signal"              # score reversed or weakened
    STOP_LOSS = "stop_loss"
    TAKE_PROFIT = "take_profit"
    TRAILING_STOP = "trailing_stop"
    TIME_EXIT = "time_exit"
    WEEKEND_CLOSE = "weekend_close"

@dataclass
class Position:
    side: PositionSide
    entry_price: float
    entry_time: datetime | None = None
    entry_bar: int = 0
    size: float = 1.0
    stop_loss: float | None = None
    take_profit: float | None = None
    trailing_stop_active: bool = False
    trailing_stop_level: float | None = None
    highest_price: float | None = None   # for long trailing stops
    lowest_price: float | None = None    # for short trailing stops
    entry_atr: float | None = None
    entry_score: float = 0.0

    def get_unrealized_pnl(self, price: float) -> float:
        return (price - self.entry_price) * self.side.value * self.size

    def check_stop_loss_hit(self, low: float, high: float) -> bool: ...
    def check_take_profit_hit(self, low: float, high: float) -> bool: ...
    def check_trailing_stop_hit(self, low: float, high: float) -> bool: ...

@dataclass
class ClosedTrade:
    side: PositionSide
    entry_time: datetime
    exit_time: datetime
    entry_price: float
    exit_price: float
    pnl: float
    exit_reason: ExitReason
```

Because none of these objects know anything about the model, each rule can be unit-tested against a synthetic price path: feed a `Position` a sequence of bar highs and lows and assert which `check_*_hit` method fires and on which bar. The `ExecutionEngine.run_backtest()` loop is the only place the two layers meet:

```
for obs, bar in zip(observations, bars):
    signal = self.signal_generator.generate(obs)        # model → score
    self.position_manager.update(bar)                   # exits: SL/TP/trail/time
    if self.position_manager.flat and self.risk.allows(bar.time):
        if abs(signal.score) >= cfg.signal.entry_threshold:
            self.position_manager.open(signal, bar)     # entry
```

3.3. Checkpoint selection with `EvalCallback`

Saving only the final training state can discard an earlier, better checkpoint. The poster pipeline therefore uses an `EvalCallback` with a separate evaluation environment and loads the best saved checkpoint for the decoupled backtest:

```
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv

train_env = SubprocVecEnv([make_env() for _ in range(8)])
eval_env  = DummyVecEnv([make_env()])           # separate, single env
callback  = EvalCallback(eval_env, best_model_save_path="best_model",
                         eval_freq=5_000,
                         n_eval_episodes=5, deterministic=True)
model = DQN("MlpPolicy", train_env, **DQN_KWARGS)
model.learn(100_000, callback=callback)
best = DQN.load("best_model/best_model")        # not `model`
```

The 2,625-trade result reported on the poster uses this selected checkpoint.

4. Results Presented at the Poster

Figure 3 reproduces the poster as presented at the SciPy 2026 virtual poster session on 2026-07-15. The systems share the 2025 data, chronological 70/30 split, 155 features and 100k training budget. The decoupled system also uses the market-only observation and direction-oriented reward described above, `EvalCallback` checkpoint selection, and the deterministic execution configuration. The results compare these two complete implementations on the same held-out period.

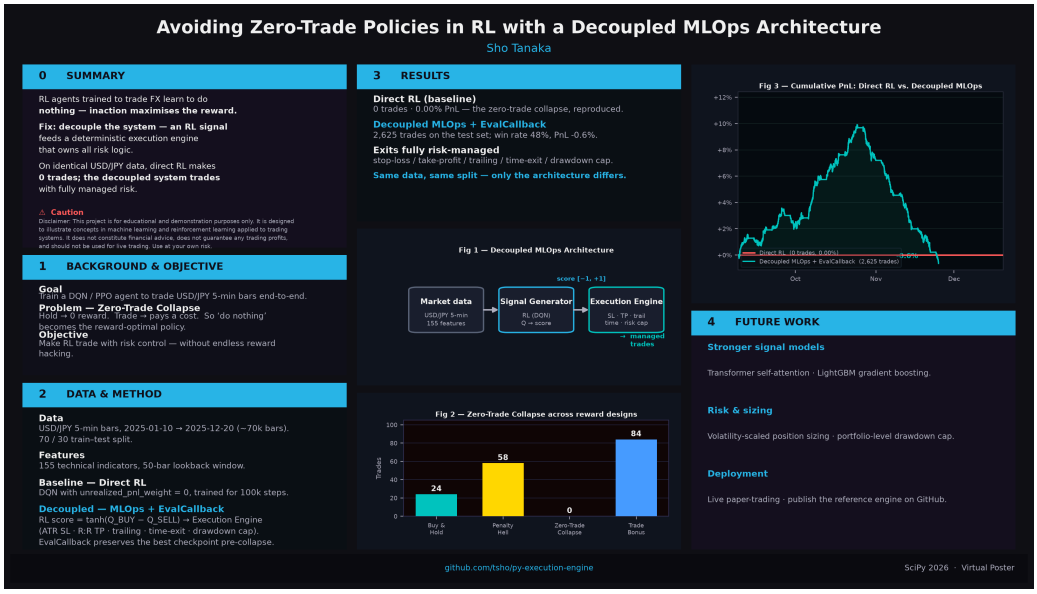


Figure 3. *The SciPy 2026 virtual poster as presented. Panel 3 and poster Fig. 3 (Figure 4) report the 2,625-trade decoupled result; poster Fig. 2 (Figure 2) shows the trade counts of the reward variants in Table 1.*

Table 2. *Test-set behaviour on USD/JPY 5-min, 2025-09-08 to 2025-12-19 (21,167 bars; 21,116 evaluable steps).*

System	Trades	Win rate	Realized PnL
Direct RL (baseline DQN)	0	—	0.00%
Decoupled + `EvalCallback`	**2,625**	**48.4%**	**-0.64%**

The decoupled system with `EvalCallback` completed 2,625 trades, with each position governed by the stop-loss, take-profit, trailing-stop, time-exit and risk rules in the execution layer. Figure 4 shows the cumulative realized PnL over the test period: the baseline is a flat line at zero, while the decoupled system rose to $+9.9\%$ in late October before giving it back to finish at -0.64%.

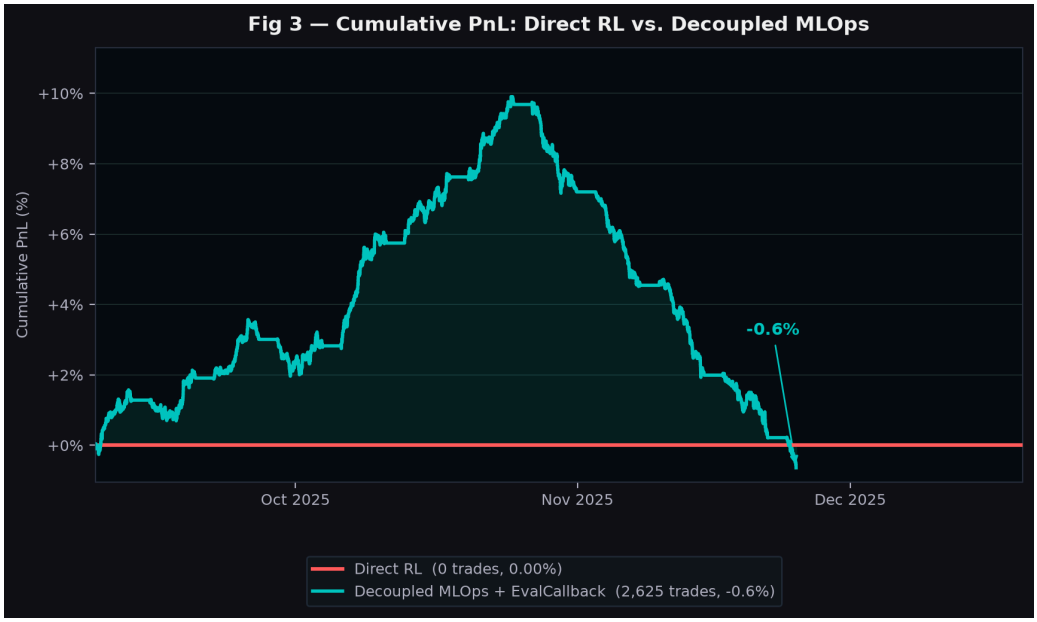


Figure 4. *Cumulative realized PnL on the 2025 test period. The direct-RL baseline completes no trades and is a flat line at 0%. The decoupled system with* `EvalCallback` *completes 2,625 trades, peaks near +10% and finishes at -0.6%. The curve characterises behaviour, not a return expectation.*

In sum, on the same data and training budget, moving position management out of the learned policy and into deterministic execution code changed the system's behaviour from completing no trades at all to completing 2,625 trades, each opened and closed by an explicit, testable rule. The near-flat final PnL underlines that the contribution is behavioural and operational—the architecture guarantees managed trading activity—not evidence that the signal itself is profitable.

5. Limitations

All results are on one currency pair and one chronological split. The reward-variant counts come from single-seed development runs and are indicative only. The direct and decoupled systems differ in observation design, reward, checkpoint selection and execution, so the comparison characterises the two complete implementations rather than isolating one architectural variable. The decoupled-system PnL is a single-seed backtest whose only trading friction is a fixed 1-pip cost deducted per completed trade. Order fills are idealized: entries fill at the bar close, and stop-loss, take-profit and trailing-stop exits fill exactly at their trigger levels. Live execution adds a variable bid-ask spread and slippage—fills worse than the trigger price when the market gaps or moves quickly—none of which is modeled here, so the reported PnL is optimistic and should not be read as an estimate of future returns.

6. Conclusion

This paper presented a decoupled response to the zero-trade failure mode observed in direct RL trading. On the same USD/JPY data and chronological split, the direct DQN completed 0 trades, whereas the decoupled signal/execution system completed 2,625 trades with every position governed by explicit risk and exit rules.

The contribution is architectural and operational: the model produces a directional score, while deterministic Python code owns position management. This boundary makes the execution rules independently testable and prevents the model from being solely respon-

sible for exits. The reported backtest does not establish profitability, but it demonstrates a practical structure for building and evaluating RL-assisted trading systems.

7. Future Work

The poster identified three directions for extending the system:

1. **Stronger signal models.** Evaluate Transformer-based models with self-attention and gradient-boosted models such as LightGBM.
2. **Risk and position sizing.** Add volatility-scaled position sizing and a portfolio-level drawdown cap.
3. **Deployment.** Validate the complete pipeline through live paper trading and publish the reference execution engine.

8. Acknowledgements and Disclosure

Portions of this work were assisted using a generative AI tool (Snowflake CoCo, Claude, Codex). The tool was used for generating and refactoring experiment and execution-engine code, for drafting and revising this manuscript, and for producing figures from the recorded result files. All experiments were run, and all outputs reviewed, verified and revised, by the author, who takes full responsibility for the accuracy and integrity of the final content.

This project is for educational and demonstration purposes only. It does not constitute financial advice, does not guarantee any trading profits, and should not be used for live trading.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[2] J. Moody, L. Wu, Y. Liao, and M. Saffell, "Performance functions and reinforcement learning for trading systems and portfolios," *Journal of Forecasting*, vol. 17, no. 5–6, pp. 441–470, 1998, [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291099-131X%281998090%2917%3A5/6%3C441%3A%3AAID-FOR707%3E3.0.CO%3B2-%23

[3] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 3, pp. 653–664, 2017, doi: 10.1109/TNNLS.2016.2522401.

[4] Farama Foundation, "Gymnasium." [Online]. Available: https://gymnasium.farama.org/

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dorber, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

[6] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[7] The Pandas Development Team, "pandas-dev/pandas: Pandas." Zenodo, 2020. doi: 10.5281/zenodo.3509134.

[8] TA-Lib Project, "TA-Lib: Technical Analysis Library." [Online]. Available: https://ta-lib.org/

[9] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.